

Task 5 -- Architectural Critique

What in nopCommerce's design helped or hindered instrumentation?

Helped. The strict layered architecture (Web -> Services -> Data -> Core) allowed placing a shared `ActivitySource` and `Meter` in a single file (`NopCommerceDiagnostics.cs`) that every service imports without circular dependencies. Well-decomposed service methods (`PlaceOrderAsync` -> `PreparePlaceOrderDetailsAsync` -> `SaveOrderDetailsAsync` -> `MoveShoppingCartItemsToOrderItemsAsync`) create natural span boundaries. The `INopStartup` interface enabled registering OpenTelemetry as a pure-additive startup class (`Order = -1`) with zero changes to existing code. ASP.NET Core's `AsyncLocal`-based `Activity` propagation meant child spans inherit parent context automatically across `await` calls.

Hindered. Services use interface-based DI but rely on `virtual` methods on concrete classes, making the decorator pattern impractical -- inline instrumentation was the only viable approach. `PlaceOrderAsync` has two control-flow paths (with/without Mutex lock), forcing duplicated metric recording. Payment plugins are opaque: spans cover only the outer `ProcessPaymentAsync` call while plugin internals remain invisible. No cross-cutting service-layer infrastructure (interceptors, middleware) exists, so each method must be instrumented individually.

What architectural changes would improve observability, and at what cost?

A **service-layer decorator pipeline** (e.g., `InstrumentedPaymentService : IPaymentService`) would auto-create spans per call, but requires maintaining wrapper classes and refactoring DI registrations. **Structured business events** at milestones (order started, payment completed) via the existing `IEventPublisher/IConsumer<T>` system would enable observability without touching service internals, but events are fire-and-forget, losing span-scoped timing. **Requiring plugins to expose an `ActivitySource`** would make their internals observable, at the cost of a breaking API change.

Where were surgical code changes necessary, and how was impact minimised?

No method signatures were changed. Instrumentation is additive: `using var activity = ...StartActivity(...)` at entry and `.Record()` before return -- both no-ops when no listener is registered. All definitions live in one shared `NopCommerceDiagnostics.cs`. PII protection uses a single `PiiSanitisationProcessor` (`BaseProcessor<Activity>`) that redacts sensitive tags at the SDK layer, requiring no per-service logic. The OTLP endpoint defaults to `localhost:4317` but is overridable via config or environment variables. Simulated failures are gated behind `OTEL_SIMULATED_FAILURE_RATE` and have zero effect when absent.